CI/CD and Containerization

Contents

CI/CD and Containerization **1**
- Scope . . . 1
- Official requirements . . . 1
- Project policy . . . 1
- Checklist . . . 2
- References . . . 2
- Related . . . 3

CI/CD and Containerization

Scope

How the project builds, how it runs locally, and what a pull request must pass.

Official requirements

- Subject IV.1: from a clone of the repository, the project's dependencies must download automatically through a Makefile or a similar mechanism.
- Subject IV.1: no third-party library source in the repository.
- Subject V.8: continuous integration is required.
- Subject V.7: the runtime environment must be specifiable, which containerisation makes precise.

Project policy

Local runtime

- **Development is fully containerised** (`brain/decisions/ADR-0029-containerised-dev-and-secret-vault.md`). A contributor needs Docker and nothing else on the host. No documented step may require a locally installed SDK, database client or CLI — anything tool-shaped is pinned in `.config/dotnet-tools.json` and restored.
- Optional compose profiles keep the everyday stack small: `vault` adds self-hosted Infisical and its dependencies, and is off by default.
- `docker compose` brings up the API and a local PostgreSQL container for development and tests. No host installation of a database engine is required — `psql` is not installed on the team's machine, and it does not need to be.
- The shared environment points at Supabase instead of the local container. Same engine, same migrations, different connection string — see `../../30-data/database-design-migrations/STANDARD.md`.
- A root `Makefile` is the documented interface:

Target	Does
`make up`	build and start the full stack
`make down`	stop and remove containers
`make migrate`	apply EF Core migrations
`make test`	run the full test suite
`make lint`	`dotnet format --verify-no-changes`
`make load`	run the load scenario

- First-run path is exactly: clone → `cp .env.example .env` → fill values → `make up`. Any extra manual step is a defect in this standard, not in the reader.

Images

- Multi-stage Dockerfile: SDK image builds, runtime image ships. The runtime image never contains the SDK or the source.
- Pin base images by tag. Do not use `latest`.
- Run as a non-root user.
- `.dockerignore` excludes `bin`, `obj`, `.git`, `node_modules`, `reference`, `.env`.
- Never bake a secret into an image layer. Secrets arrive as environment variables at run time.

Pipeline

Every pull request runs, in order, and all must pass:

1. `dotnet restore`
2. `dotnet format --verify-no-changes`
3. `dotnet build -warnaserror` — this is where the analyzer size and complexity limits fire (`dotnet-csharp` standard, `.editorconfig`)
4. `dotnet test` including container-backed integration tests
5. `dotnet list package --vulnerable --include-transitive` — fail on high severity
6. OpenAPI document generation — fail if it does not build

A red pipeline blocks merge. Disabling a check to get a merge through is not an option; fix the cause or revert.

Environments

- Configuration comes from environment variables in every environment (twelve-factor).
- `Development` enables the interactive OpenAPI UI and detailed errors. No other environment does.
- Database migrations are applied as an explicit step, never automatically on application start in a shared environment.

Checklist

☐ `make up` works from a fresh clone
☐ Multi-stage Dockerfile, pinned base, non-root user
☐ `.dockerignore` excludes secrets and build output
☐ CI runs format, build, test, vulnerability scan, OpenAPI generation
☐ Analyzer packages restored so the size and complexity limits actually run in CI
☐ No secret in any image layer or CI log

References

- .NET container images — https://learn.microsoft.com/dotnet/core/docker/build-container
- Docker Compose — https://docs.docker.com/compose/

- Twelve-Factor App — https://12factor.net/

Related

`../../00-foundations/engineering-baseline/STANDARD.md` · `../../60-quality/testing/STANDARD.md` · `../../40-security/secrets-management/STANDARD.md`